

Raw Data

Telugu and English SRTs with different srt ids.

1) Data Preprocessing

The folder contains Telugu and English SRTS.

2.py (Telugu Merger)

Rules:

- **To detect telugu srt from each folder:** Uses Unicode range [\u0C00-\u0C7F] with a 50-character threshold.
- **Comma-based Merge:** If a Telugu block ends in a comma (,), it merges with the following block.

We don't follow ending punctuation logic, as there is extra telugu content with no punctuation. Ex: Information on the movie screen is written in telugu srts and ignored in english. So, to stop merging the noise without punctuation into next lines, we only merge if the last character of the telugu line is a comma.

- **Timestamp Calculation:** Start time is taken from the first block; End time from the second.

3.py (English Cleaning & Sentence Merger)

Rules:

- **Tag Removal:** Strips <i> and content inside flower braces { }. Ex: {\an8}
- **Sentence Completion:** Merges sequential English blocks if they do **not** end in ., !, or ?.
- **Exclusion:** Blocks containing only [brackets] are ignored and treated as separate units.

4.py (Alignment)

Rules:

Telugu and English subtitles are aligned using time overlap

- Overlap is detected when two subtitle time ranges intersect using condition:
 - $\text{English}(\text{start time}) < \text{Telugu}(\text{end time})$ and $\text{English}(\text{end time}) > \text{Telugu}(\text{start time})$
- This allows:

- One Telugu line mapping to multiple English lines.
- Multiple Telugu lines mapping to one English line.
- All overlapping subtitles are merged into a single block and subtitles are sorted by start time and consecutive blocks with identical English text are merged(rewrite).
- Telugu subtitles with no English overlap are marked as: NO ENGLISH MATCH FOUND
- English subtitles with no Telugu overlap are marked as: NO TELUGU MATCH FOUND 605 manual

5.py

Rules:

- **Filler Removal:** Strips words like Mmm, Hmm, Uh-huh, Ah,Oh if they are followed by an ending punctuation.
- Example:

Here, "Oh." is removed, as telugu translation isn't given and the output results in equal number of ending punctuations in each side so it can be split in the further steps.

"79",

"-సరే. -హానికరం కాని బుల్లెట్. సరే, వాడు ఒక కిలో ఎత్తేశాడు.",

"Oh. Okay. - Bean bag. - Okay, he clipped the key."

- **Standardization:** Converts ... to ... and cleans up leading/trailing hyphens.
- **Clean and Tagged versions:** 2 separate versions are created from here one version with tags and another where all emotion tags are removed.

6.py

The Goal: Automatically fix Telugu sentences that are missing ending punctuation.

- **Preprocessing:** Identifies the English punctuation mark (., !, ?) at the end of the English segment (ignoring tags like [bracketed text]).
-
- **Insertion Logic:** It inserts the English punctuation immediately after that character, even if there are trailing tags.
 - **Ex:** Telugu: ఎక్కడికి వెళ్ళాడు | English: Where did he go?
 - **Result:** ఎక్కడికి వెళ్ళాడు?

7.py (Identify and flag ending punctuation mismatch to split into lines)

The Goal: A check to find rows where the number of ending punctuation doesn't match.

- Full stops in examples like (Dr., Mr., a.m., etc.) aren't counted as "sentence ends."
- Ignores dots following a Telugu character (like కె.ఆర్. రావు) to prevent false mismatch flags.
- **Rule 1:** Ignore as mismatch if either punctuation count is 1. (Only ? and . are considered as punctuations here) as we can't further split if it's only 1.
- The row is flagged if Telugu punctuation count > 1 and English punctuation count > 1.
- **Rule 2:** Ignore count of ? if it is the first punctuation and not followed by another ?.
- **Mechanism:** The function looks for the first ?. If both sides have one, it "subtracts" that from the count to see if the remaining dots match.

8.py (Strict Audit)

- **Logic:** It performs a raw count of every . and ? in both cleaned strings.
- **Output:** Generates a strict_audit_report.csv. If English has 2 dots and Telugu has 1, it is flagged as STRICT_MISMATCH. This forces a manual review to ensure no sentence was accidentally merged or deleted during the translation/cleaning process.

9.py (Segmentation)

1. Rule: Split into separate sentences only if the number of punctuations matches on both sides.

2.

- **Rule:** If the punctuation before the question mark is the same, then split.
- **Rule:** Split if the punctuation before the ? is identical and not followed by another ?.
- **Mechanism:** The script finds the index of the first ?. It checks all punctuation *before* that index. If English and Telugu both have, for example, two periods followed by a question mark (. . ?), it splits those three sentences and groups the remaining "mismatched" text into a single final row.

10.py (Master Audit Reporter)

- **Index Scanning:** Parses structural audit logs for rows flagged by length or punctuation mismatches.
- **Content Extraction:** Pulls corresponding Telugu and English text segments from the segmented data.

- **Centralized Output:** Generates a single CSV in the central audit directory to provide full context for manual correction.

MANUAL SPLITTING OF PUNCTUATION MISMATCHES IS DONE.

బయట ఉన్నా, జైల్లో ఉన్నా, పోలీసులు ఒకరినొకరు
కాపాడుకుంటారు. అది పోలీసుల మధ్య ఉండే బంధం.

Whether inside, outside. Blue is blue. It's a
brotherhood.

లేదు. అస్సలు లేదు.

Am I connected? No. No.

11.py

- This stage integrates manual human corrections back into the respective position in their csvs.

So, after all these steps the output data looks like:

```
"116", "మెనూ చూశారా, సర్?", "Have you had a look at the menu, sir?"
"118", "అయ్యో, ఇంకా చూడలేదు.", "Jesus. No."
"118", "మన్నించండి రోషల్.", "I'm sorry Rochelle."
"119", "నాకు మంచి నీరు తీసుకువస్తారా?", "Could I get a water, please?"
```